

VITALSYNC

NEXT-GENERATION HEALTHCARE MANAGEMENT PLATFORM

Technical Documentation • Deployment Guide • Architecture Reference

Gaurav Tiwari — 2026

1. PROJECT OVERVIEW

VitalSync is a full-stack healthcare management platform built on Next.js 14. It connects patients, doctors, and hospitals through a single unified interface — handling everything from appointment booking and OPD queue management to real-time video consultations and AI-powered medical triage.

What It Does

- ▶ Patients can register, search for doctors by symptom (multilingual AI triage), book online and offline appointments, and manage their full medical history.
- ▶ Doctors are assigned to departments within hospitals, manage consultation fees, handle queues, and conduct video calls via LiveKit.
- ▶ Hospitals manage bed allocation, departments, OPD queues, billing (Razorpay), and room history.
- ▶ An AI assistant (Curo) powered by Gemini API provides two tiers of medical guidance: CuroBeat (premium, structured) and CuroFlash (free, streamed).
- ▶ Real-time features (queue updates, notifications) are powered by Pusher.
- ▶ File uploads (profile images, medical documents) are handled by EdgeStore.
- ▶ Authentication is handled separately for each role (Patient, Doctor, Hospital) using Lucia Auth with Prisma adapter. Google OAuth is also supported for patients.

Author

Name: Gaurav Tiwari

Version: 0.1.0 (2026)

License: MIT

Stack: Next.js 14, TypeScript, Prisma ORM, PostgreSQL (Neon), Lucia Auth, LiveKit, Pusher, Razorpay, Gemini AI

2. ARCHITECTURE & PROJECT STRUCTURE

The project follows Next.js App Router conventions. All server logic lives in route handlers (app/api/**) or server actions. Client components are kept minimal.

LAYER	TECHNOLOGY	PURPOSE
Frontend	Next.js 14 App Router + TypeScript	Server and Client Components, routing
UI Library	ShadCN UI + Radix Primitives + Tailwind CSS	Accessible, composable component system
Animation	Framer Motion (motion)	Page transitions and UI animations
Database ORM	Prisma 5 + PostgreSQL	Type-safe DB queries, schema-first
Database Host	Neon (Serverless PostgreSQL)	Auto-scaling, branch-based DB for Vercel
Authentication	Lucia Auth v3 + Prisma Adapter	Session-based auth for 3 user roles
Google OAuth	Custom OAuth flow (api/auth/google)	Patient login via Google account
Video Calls	LiveKit SDK + Components	Secure WebRTC video consultations
Realtime	Pusher (Server + Client)	Queue updates, live notifications
AI / Triage	Gemini API (@google/generative-ai)	Symptom triage, AI chat (Curo)
Payments	Razorpay	Online billing, order creation, verification
File Storage	EdgeStore	Image and document uploads
Deployment	Vercel (Serverless)	Edge-optimized Next.js hosting

Key Directories

PATH	WHAT LIVES HERE
app/ (main) /	Auth pages for Patient, Doctor, Hospital
app/api/	All REST endpoints (billing, rooms, search, auth, etc.)
app/about-us/assistantBhura/	Curo AI assistant page and shimmer loading
lib/	Shared utilities: db.ts (Prisma), lucia.ts, gemini.ts, pusher.ts
prisma/schema.prisma	Full database schema (13 models)
prisma/migrations/	7 migration files covering all schema versions
components/	Shared React components and all ShadCN UI primitives
hooks/	Custom hooks: useLocation, useLoggedIn, useMediaQuery, etc.
styles/	CSS modules for global and component-level styles
public/	Static assets, SVGs, feature images

3. DATABASE SCHEMA

Prisma schema targets PostgreSQL (Neon). The schema has 13 models across three user domains. All relationships use cascade deletes.

MODEL	PRIMARY KEY	KEY RELATIONS / NOTES
Patient	cuid()	Has MedicalHistory[], PatientSession[], Transaction[], OnlineMeet[], OfflineMeet[]
MedicalHistory	String (manual)	Belongs to Patient (cascade delete)
PatientSession	String (manual)	Lucia session — belongs to Patient
Hospital	cuid()	Has Department[], Room[], HospitalRoomHistory[], HospitalSession[]
Room	cuid()	Belongs to Hospital. Tracks type (ICU, ward, etc.) and booking state
HospitalRoomHistory	cuid()	Immutable audit log of room check-in/check-out events
Department	cuid()	Belongs to Hospital. Has DoctorinDept[] (many-to-many bridge)
Doctor	cuid()	userId is unique login ID. Has DoctorinDept[], DoctorSession[]
DoctorinDept	Composite (deptId, doctorId)	Many-to-many bridge: Doctor in Department. Stores fees, availability
OnlineMeet	cuid()	Scheduled video appointment. Links Patient + Doctor + Department
OfflineMeet	cuid()	Scheduled in-person appointment. Same structure as OnlineMeet
Transaction	cuid()	Razorpay transaction record linked to Patient
HospitalSession	String (manual)	Lucia session for Hospital users
DoctorSession	String (manual)	Lucia session for Doctor users
Feedback	cuid()	User feedback form submissions. Has Reply[]
Reply	cuid()	Admin replies to Feedback (cascade delete)

4. ENVIRONMENT VARIABLES

All secrets are loaded via environment variables. Never commit real values to git. The .gitignore already excludes .env*.local. Add these as Environment Variables in the Vercel dashboard for production.

4.1 Database (Neon PostgreSQL)

VARIABLE	DESCRIPTION	EXAMPLE / SOURCE
DATABASE_URL	Pooled connection string (pgbouncer). Used by Prisma at runtime on Vercel serverless functions.	<code>postgresql://user:pass@ep-xxx.region.aws.neon.tech/vitalsync?sslmode=require</code>
DIRECT_URL	Direct (unpooled) connection. Used only during prisma migrate deploy. Do NOT set sslmode=require on this one.	<code>postgresql://user:pass@ep-xxx-direct.region.aws.neon.tech/vitalsync</code>

4.2 LiveKit (Video Rooms)

VARIABLE	DESCRIPTION	EXAMPLE / SOURCE
LIVEKIT_API_KEY	Server-side API key for creating room tokens.	<code>APISomeKey123</code>
LIVEKIT_API_SECRET	Server-side secret paired with the API key.	<code>SecretString456</code>
LIVEKIT_URL	Your LiveKit Cloud project URL.	<code>wss://myproject.livekit.cloud</code>
NEXT_PUBLIC_CONN_DETAILS_ENDPOINT	API route that returns connection details for client SDK.	<code>/api/connection-details</code>
NEXT_PUBLIC_LK_RECORD_ENDPOINT	Optional recording endpoint URL.	<code>https://your-recording-endpoint</code>
NEXT_PUBLIC_SHOW_SETTINGS_MENU	Toggle LiveKit settings menu in UI.	<code>true</code>

4.3 Gemini AI

VARIABLE	DESCRIPTION	EXAMPLE / SOURCE
NEXT_PUBLIC_GEMINI_API_KEY	API key for Google Generative AI. Prefixed NEXT_PUBLIC_ so client can call it directly. Consider moving to server-only if security is a concern.	<code>AIzaSy...</code>

4.4 Pusher (Realtime)

VARIABLE	DESCRIPTION	EXAMPLE / SOURCE
NEXT_PUBLIC_PUSHER_APP_ID	Pusher app ID (public).	1234567
NEXT_PUBLIC_PUSHER_PUBLISHABLE_KEY	Pusher publishable key (public).	abc123def456
PUSHER_SECRET_KEY	Pusher secret key (server-only).	secretkey789
NEXT_PUBLIC_PUSHER_CLUSTER	Pusher cluster region.	ap2

4.5 Razorpay (Billing)

VARIABLE	DESCRIPTION	EXAMPLE / SOURCE
key_id	Razorpay Key ID (use test key for staging).	rzp_test_xxxx
key_secret	Razorpay Key Secret.	secretvalue

4.6 Application & Google OAuth

VARIABLE	DESCRIPTION	EXAMPLE / SOURCE
NEXT_PUBLIC_APP_URL	Public URL of the deployed app. Used for OAuth redirects and absolute URLs.	https://vitalsync.vercel.app
GOOGLE_CLIENT_ID	Google OAuth2 client ID from Google Cloud Console.	xxxx.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET	Google OAuth2 client secret.	GOCSPX-...
GOOGLE_REDIRECT_URI	Must exactly match the redirect URI in Google Cloud Console.	https://vitalsync.vercel.app/api/auth/google/callback

5. DEPLOYMENT GUIDE: VERCEL + NEON

This guide takes you from a fresh git repository to a live Vercel deployment with a Neon PostgreSQL database. Follow each step in order.

Phase A: Set Up Neon Database

STEP 1 Create a Neon Project

Go to <https://neon.tech> and sign up or log in.

Create a new project. Name it 'vitalsync' (or any name).

Select a region close to your users (ap-south-1 for India).

Neon will create a default database called 'neondb' — rename it to 'vitalsync' in Settings if desired.

STEP 2 Get Your Connection Strings

In Neon Dashboard, go to your project > Connection Details.

Copy the Pooled connection string (pgbouncer=true) — this is your DATABASE_URL.

Copy the Direct connection string (no pgbouncer) — this is your DIRECT_URL.

Add ?sslmode=require to the DATABASE_URL if not already present.

STEP 3 Run Database Migrations

Locally, set your .env.local with the real Neon connection strings, then run:

```
$ npx prisma migrate deploy
```

This applies all 7 existing migrations to your Neon database.

Verify in Neon Dashboard > Tables that all 13 models appear.

Phase B: Push Code to GitHub

STEP 4 Initialize Git Repository

```
$ git init
$ git add .
$ git commit -m "Initial commit: VitalSync"
```

Create a new repo on GitHub, then push:

```
$ git remote add origin https://github.com/yourusername/vitalsync.git
$ git push -u origin main
```

Ensure .env.local is NOT committed (.gitignore already excludes it).

Phase C: Deploy on Vercel

STEP 5 Import Project on Vercel

Go to <https://vercel.com> and log in.

Click 'Add New > Project' and import your GitHub repository.

Vercel auto-detects Next.js. Do NOT change the framework preset.

Set the build command to: prisma generate && next build

Set the install command to: npm install

STEP 6 Add Environment Variables

Before clicking Deploy, expand 'Environment Variables'.

Add every variable from Section 4 of this document.

For DATABASE_URL: paste the pooled Neon connection string.

For DIRECT_URL: paste the direct Neon connection string.

For NEXT_PUBLIC_APP_URL: you will need to deploy once first to get the URL, then update this variable with your actual Vercel URL.
Set the same variables for Production, Preview, and Development environments.

STEP 7 Deploy

Click 'Deploy'. Vercel will run: npm install > prisma generate > next build.

Monitor the build logs. Common failure points:

- Missing environment variable: check all vars are set
- prisma generate fails: ensure postinstall script is in package.json (it is)
- Module not found: check node_modules are not committed

STEP 8 Update Google OAuth Redirect URI

After first deploy, get your actual Vercel URL (e.g. vitalsync.vercel.app).

Go to Google Cloud Console > APIs & Services > Credentials.

Edit your OAuth 2.0 client and add the authorized redirect URI:

<https://vitalsync.vercel.app/api/auth/google/callback>

Update GOOGLE_REDIRECT_URI and NEXT_PUBLIC_APP_URL in Vercel dashboard.

Redeploy (Vercel > Deployments > Redeploy).

STEP 9 Verify Deployment

Visit your Vercel URL. Test each user type registration (Patient, Doctor, Hospital).

Test AI triage (requires valid GEMINI_API_KEY).

Test video room creation (requires LiveKit credentials).

Test payment flow with Razorpay test mode keys.

Check Neon Dashboard > Monitoring to confirm DB queries are arriving.

6. KNOWN ISSUES & IMPORTANT NOTES

Security Concerns

- ▶ NEXT_PUBLIC_GEMINI_API_KEY is exposed to the browser. If your Gemini API has no IP restrictions, anyone can use your quota. Consider moving Gemini calls server-side.
- ▶ key_id (Razorpay) does not have NEXT_PUBLIC_ prefix but is used in a client-facing context in some billing routes — audit before going live.
- ▶ The Lucia middleware only protects /patient-dash routes. Hospital and Doctor dashboard routes have no middleware protection — rely on cookie checks inside page components. Audit this before production.

Neon + Prisma Specifics

- ▶ Neon's serverless driver is NOT used here — the project uses standard Prisma with DATABASE_URL pointing to Neon's pgbouncer endpoint. This is correct for Vercel serverless functions.
- ▶ The DIRECT_URL is required for prisma migrate deploy. Without it, migrations may fail due to pgbouncer transaction mode limitations.
- ▶ previewFeatures = ["relationJoins"] in the Prisma schema requires Prisma 5.0+. The project is on 5.18.0 — no issue.

Vercel Specifics

- ▶ argon2 (used for password hashing) requires native binaries. next.config.mjs already externalizes @node-rs/argon2 and @node-rs/bcrypt for Webpack — do not remove this.
- ▶ net package is listed as a dependency in package.json. This is a Node.js built-in and should not be in dependencies. It will not cause issues but is unnecessary.
- ▶ tsconfig.tsbuildinfo (477KB) is committed to the repo. This is safe but adds unnecessary size. Consider adding it to .gitignore.
- ▶ temp.py in the project root appears to be a development script. Remove it before production or add to .gitignore.

External Services Required

SERVICE	FREE TIER AVAILABLE?	NOTES
Neon	Yes (0.5GB storage, 1 compute)	More than enough for development
Vercel	Yes (Hobby plan)	Sufficient for personal/demo projects
Pusher	Yes (200k msgs/day, 100 connections)	Fine for low-traffic apps
LiveKit	Yes (LiveKit Cloud free tier)	Check concurrent room limits
Gemini API	Yes (generous free quota)	Rate limits apply
Razorpay	Test mode is free	Need KYC for live payments in India
EdgeStore	Yes (1GB storage)	Suitable for small apps
Google OAuth	Free	Requires verified Google Cloud project

7. API ROUTES REFERENCE

All routes live under `app/api/`. They are Next.js Route Handlers (not Express). All database access uses the singleton Prisma client from `lib/db.ts`.

ROUTE	METHOD(S)	PURPOSE
<code>/api/auth/google</code>	GET	Initiates Google OAuth2 flow, redirects to Google consent screen
<code>/api/auth/google/callback</code>	GET	Handles Google OAuth callback, creates/updates Patient, sets Lucia session
<code>/api/billing/doctor</code>	GET	Fetches billing info for a doctor
<code>/api/billing/order</code>	POST	Creates a new Razorpay order
<code>/api/billing/patient</code>	GET	Fetches billing info for a patient
<code>/api/billing/updateTransaction</code>	POST	Updates transaction after successful Razorpay payment
<code>/api/billing/updateOfflineTransaction</code>	POST	Updates offline/manual payment transaction
<code>/api/billing/verify</code>	POST	Verifies Razorpay payment signature
<code>/api/chartdata</code>	GET	Returns chart data for hospital dashboard analytics
<code>/api/connection-details</code>	GET	Returns LiveKit connection details (room token) for video call
<code>/api/contact/feedback</code>	POST	Submits user feedback to Feedback table
<code>/api/createroom/[id]</code>	POST	Creates a new room for a hospital
<code>/api/dept</code>	GET/POST	Lists or creates hospital departments
<code>/api/doctodept/[did]</code>	GET	Fetches departments and doctors for a given doctor ID
<code>/api/doctor/[id]</code>	GET	Fetches a single doctor's public profile
<code>/api/editroom/[id]</code>	PUT	Updates room details (type, availability, booking)
<code>/api/edgestore/[...edgestore]</code>	ALL	EdgeStore SDK route handler for file uploads
<code>/api/essentialHospitalDetails/[id]</code>	GET	Fetches essential details for hospital card display
<code>/api/fetchDoctorCardData/[id]</code>	GET	Fetches doctor card data for display
<code>/api/hospital/[id]</code>	GET	Fetches full hospital profile
<code>/api/patient/[id]</code>	GET	Fetches patient profile by ID
<code>/api/queueOfDoctor/[id]/[deptId]</code>	GET	Gets the current queue for a doctor in a department
<code>/api/queueOfHospital/[id]</code>	GET	Gets all queues for a hospital
<code>/api/reply/[id]</code>	POST	Admin replies to a feedback message
<code>/api/roomhis/patientdata/[id]</code>	GET	Gets patient room history
<code>/api/search</code>	GET	AI-powered semantic search (Gemini) for doctors/hospitals by symptom
<code>/api/test</code>	GET	Dev/debug endpoint — remove before production
<code>/api/totalrooms/[id]</code>	GET	Gets total room counts/availability for a hospital

8. QUICK REFERENCE COMMANDS

Local Development

Install dependencies	<code>\$ npm install</code>
Run dev server	<code>\$ npm run dev</code>
Open browser	<code>http://localhost:3000</code>
Generate Prisma client	<code>\$ npx prisma generate</code>
Push schema (dev, no migration)	<code>\$ npx prisma db push</code>
Create new migration	<code>\$ npx prisma migrate dev --name <name></code>
Apply migrations to production DB	<code>\$ npx prisma migrate deploy</code>
Open Prisma Studio	<code>\$ npx prisma studio</code>

Vercel CLI (Optional)

Install Vercel CLI	<code>\$ npm install -g vercel</code>
Login to Vercel	<code>\$ vercel login</code>
Deploy to preview	<code>\$ vercel</code>
Deploy to production	<code>\$ vercel --prod</code>
Pull env vars locally	<code>\$ vercel env pull .env.local</code>
View deployment logs	<code>\$ vercel logs</code>

VitalSync — Built with purpose.

Gaurav Tiwari · 2026 · MIT License